

Testing Y QA

Parte 8: Investigación de participacion de testing en metodologias TDD, BDD y ATDD

Integrantes:

Romina Alejandra Diaz

Teodoro Welyczko

Francisco Daurat

Mateo Colombo

Ignacio Lorente Grignola

Profesor: Gabriel Hugo Taboada

Prólogo

Prólogo.....	2
Enunciado.....	3
Consigna general de la actividad.....	3
Resolución de consigna.....	3
Demostración con aplicación TDD.....	3
Enlaces e información importante:.....	6

Enunciado

Consigna general de la actividad

Realizar una investigación breve, en base al contenido del módulo y otras fuentes confiables, sobre cómo participa el testing en las metodologías TDD, BDD y ATDD.

Responder a la siguiente pregunta: ¿cómo participa el testing en las técnicas de TDD, BDD y ATDD?

Resolución de consigna

El testing ocupa un rol central en las metodologías ágiles modernas como TDD (Test Driven Development), BDD (Behavior Driven Development) y ATDD (Acceptance Test Driven Development). En todos los casos, las pruebas dejan de ser una actividad final para transformarse en una práctica integrada al desarrollo desde el inicio. Según el material de clase, en metodologías ágiles “las pruebas se realizan en ciclos cortos e integradas al desarrollo” y se busca asegurar calidad continua durante todo el proceso.

En **TDD**, primero se escribe una prueba automatizada que falla, luego el código mínimo para pasarla y finalmente se refactoriza. El testing guía el diseño técnico y permite detectar errores tempranamente. Es común utilizar herramientas como JUnit en Java o RSpec en Ruby. Por ejemplo, antes de implementar un método que calcule descuentos, el desarrollador crea una prueba que valide el resultado esperado para distintos porcentajes.

En **BDD**, el foco está en el comportamiento esperado del sistema desde la perspectiva del negocio. Las pruebas se escriben en lenguaje natural mediante escenarios del tipo Given-When-Then, facilitando la comunicación entre desarrolladores, testers y clientes. Herramientas como Cucumber permiten automatizar estos escenarios. Por ejemplo: “Dado un usuario autenticado, cuando realiza una compra, entonces el sistema debe generar la factura”.

Por su parte, **ATDD** define pruebas de aceptación antes del desarrollo, basadas en criterios acordados con el cliente o Product Owner. Estas pruebas validan que el sistema cumpla los requisitos funcionales del negocio y suelen automatizarse para ejecutarse continuamente.

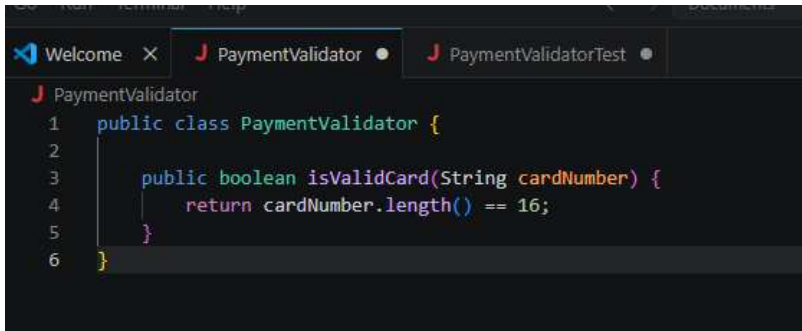
En conclusión, el testing en TDD, BDD y ATDD no solo verifica calidad, sino que guía el desarrollo, mejora la comunicación del equipo y reduce errores en etapas avanzadas del proyecto.

Demostración con aplicación TDD

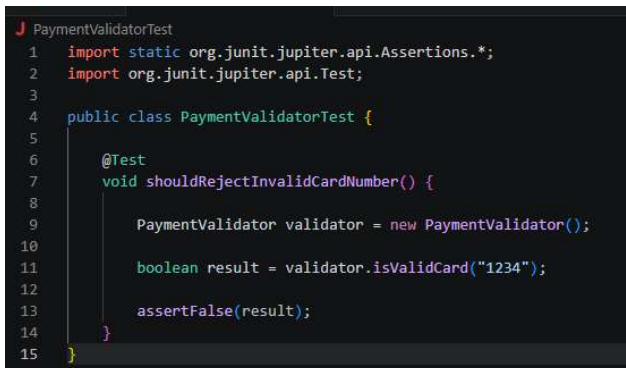
Se realizó una demostración práctica de TDD utilizando JUnit sobre una validación del módulo de **pagos del e-commerce**. Primero se escribió una prueba automatizada para validar el rechazo de tarjetas inválidas. Luego se implementó el código mínimo necesario para satisfacer la prueba y

finalmente se refactorizó la solución, manteniendo todas las pruebas exitosas. Esta práctica permitió demostrar el ciclo **RED-GREEN-REFACTOR** característico de TDD y su integración con las pruebas unitarias definidas en el Plan de Testing

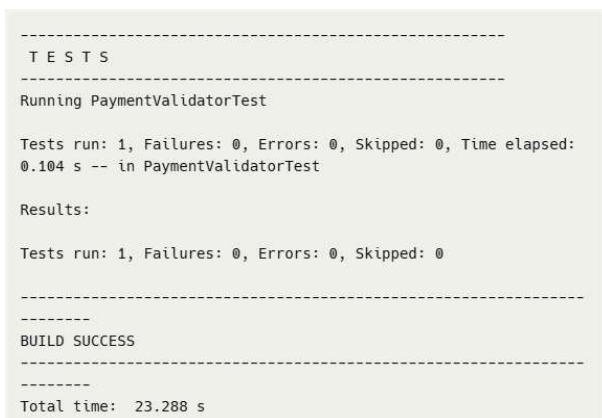
Ejecución fase **GREEN**



```
1 public class PaymentValidator {
2
3     public boolean isValidCard(String cardNumber) {
4         return cardNumber.length() == 16;
5     }
6 }
```



```
1 import static org.junit.jupiter.api.Assertions.*;
2 import org.junit.jupiter.api.Test;
3
4 public class PaymentValidatorTest {
5
6     @Test
7     void shouldRejectInvalidCardNumber() {
8
9         PaymentValidator validator = new PaymentValidator();
10
11         boolean result = validator.isValidCard("1234");
12
13         assertFalse(result);
14     }
15 }
```



```
-----
T E S T S
-----
Running PaymentValidatorTest

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.104 s -- in PaymentValidatorTest

Results:

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

-----
BUILD SUCCESS
-----
Total time: 23.288 s
```

Ejecución fase **RED**

```
J PaymentValidatorTest
1  import static org.junit.jupiter.api.Assertions.*;
2  import org.junit.jupiter.api.Test;
3
4  public class PaymentValidatorTest {
5
6      @Test
7      void shouldRejectInvalidCardNumber() {
8
9          PaymentValidator validator = new PaymentValidator();
10
11          boolean result = validator.isValidCard("1234");
12
13          assertFalse(result);
14      }
15  }
```

```
Welcome X J PaymentValidator J PaymentValidatorTest
J PaymentValidator
1  public class PaymentValidator {
2
3      public boolean isValidCard(String cardNumber) {
4          return true;
5      }
6  }
```

```
-----
T E S T S
-----
Running PaymentValidatorTest

Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE!

PaymentValidatorTest.shouldRejectInvalidCardNumber <<< FAILURE!

org.opentest4j.AssertionFailedError: expected: <false> but was:
<true>
```

Corrección de código con la solución mínima para que pase:

```
Welcome X J PaymentValidator J PaymentValidatorTest
J PaymentValidator
1  public class PaymentValidator {
2
3      public boolean isValidCard(String cardNumber) {
4          return cardNumber.length() == 16;
5      }
6  }
```

```
-----  
T E S T S  
-----  
Running PaymentValidatorTest  
  
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:  
0.104 s -- in PaymentValidatorTest  
  
Results:  
  
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0  
  
-----  
BUILD SUCCESS  
-----  
Total time: 23.288 s
```

Enlaces e información importante:

Aquí compartimos 3 enlaces en los cuales se explica aparte de la teoría vista en clase el testing en TDD, BDD y ATDD, imágenes representativas de cómo el testing se involucra y como se puede aplicar cada uno con ejemplos.

<https://cleancodeguy.com/es/blog/red-green-refactor-tdd>

<https://cucumber.io/docs/gherkin/reference/>

<https://www.businessanalysisexperts.com/gherkin-user-stories-given-when-then-examples/>